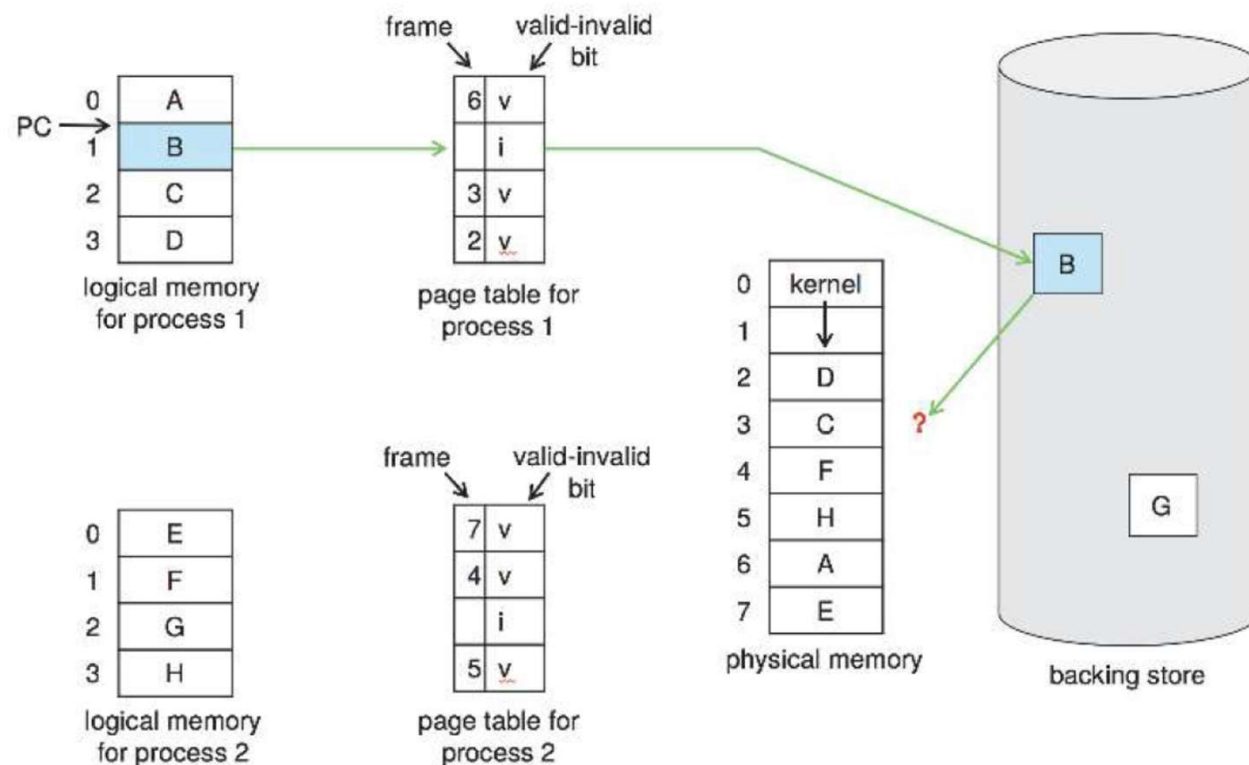


Sostituzione delle Pagine

- ❑ Se un processo di 10 pagine e ne usa la metà allora risparmiati 5 accessi
- ❑ Si può aumentare il grado di multiprogrammazione del doppio
- ❑ Si sta **sovrallocando** memoria
- ❑ ... ma se non c'è un frame libero dopo il page-fault?

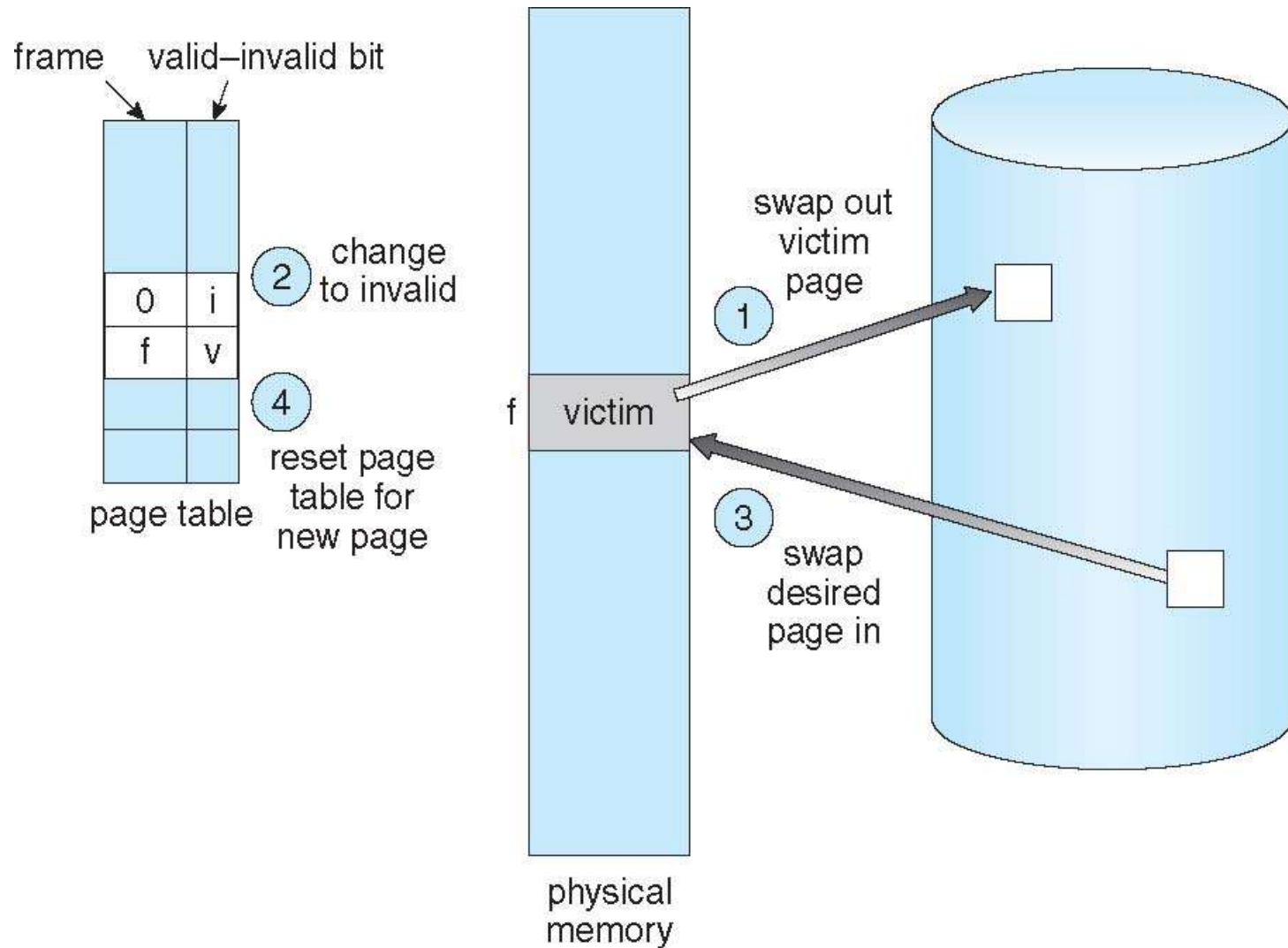


Sostituzione delle Pagine

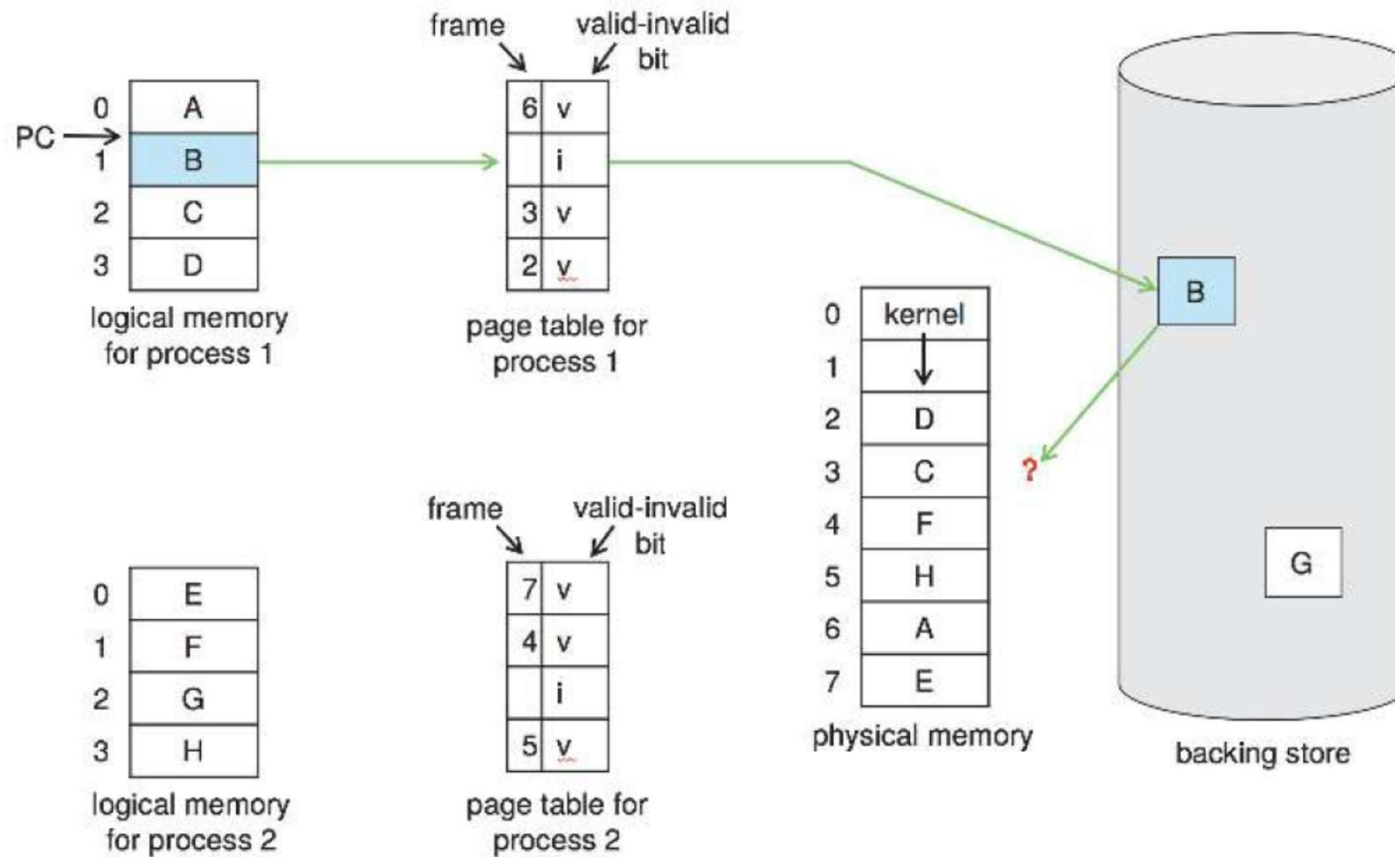
- Se un processo di 10 pagine e ne usa la metà allora risparmiati 5 accessi
- Si può aumentare il grado di multiprogrammazione del doppio
- Si sta **sovrallocando** memoria

- Se non c'è un frame libero dopo il page-fault?
 - SO può terminare il processo (non è la scelta migliore, utente non dovrebbe accorgersi della gestione)
 - SO può fare uno swapping di un intero processo riducendo il livello di multiprogrammazione (ma non si usa perché troppo costoso)
 - SO può fare **page replacement** combinando swapping e paging

Sostituzione di Pagine



Bisogno della sostituzione di pagine



Sostituzione delle Pagine

- La sostituzione di pagine è fondamentale per il demand paging
 - completa la separazione tra la memoria logica e fisica – una larga memoria virtuale può essere ottenuta da una memoria fisica più piccola
- Previene la **sovrallocazione** di memoria modificando il servizio di page-fault per includere la sostituzione di pagine
- Usa **bit di modifica** (**dirty bit**) per ridurre l'overhead del trasferimento di pagine – solo le pagine modificate sono scritte su disco

Sostituzione di Pagine

1. Trova la locazione della pagina desiderata su disco
2. Trova un frame libero:
 - se c'è un frame libero usalo
 - se non c'è usa algoritmo di page replacement per selezionare un **frame vittima**
 - scrivi il frame vittima su disco se il dirty bit è impostato
3. Porta la pagina desiderata nel nuovo frame libero; aggiorna la tabella di pagina e dei frame
4. Continua il processo riavviando l'istruzione che ha causato il trap

Nota: potenzialmente 2 transferimenti di pagina per page fault

- Incrementato Effective Access Time (EAT)
- Utilizzando il bit di modifica si può ridurre ad un accesso (2 solo se modifica)

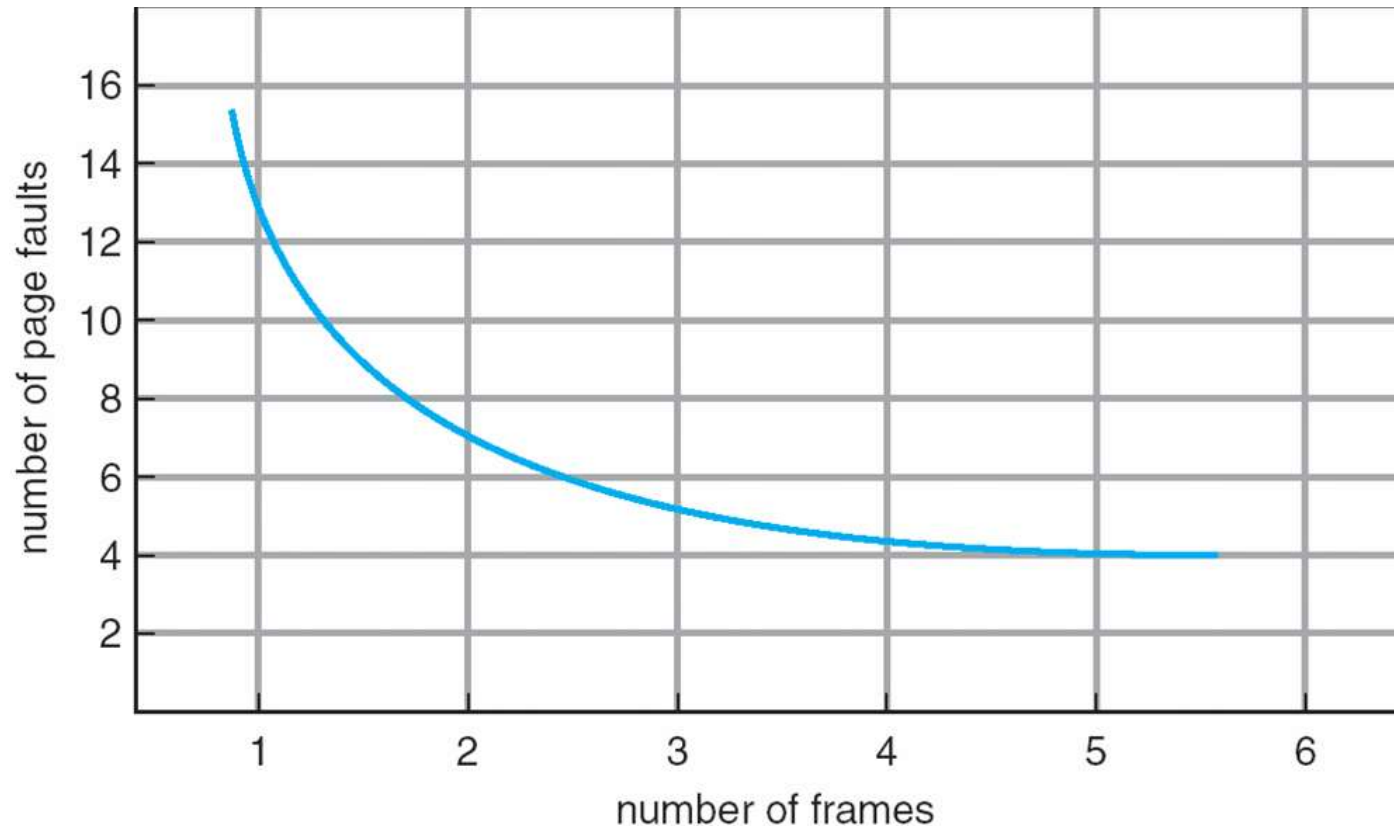
Algoritmi di sostituzione di pagina e di frame

- Per implementare il demand paging occorre:
 - **Algoritmo di allocazione frame**
 - ▶ Se più processi, quanti frame assegnare ad ogni processo?
 - **Algoritmo di sostituzione pagina**
 - ▶ Se sostituzione pagina quali frame sostituire?
 - ▶ Si vuole mantenere un basso tasso di page-fault sia per il primo accesso che per il riaccesso (accesso I/O costoso anche il minimo miglioramento è importante)

Algoritmi di sostituzione di pagina e di frame

- Per implementare il demand paging occorre:
 - **Algoritmo di allocazione frame**
 - **Algoritmo di sostituzione pagina**
- Algoritmo valutato su una particolare **stringa dei riferimenti** in memoria (reference string) calcolando il numero di page fault generati
 - Riferimenti generati sinteticamente o registrando accessi
 - Stringa indica solo numeri di pagina, non indirizzo completo
 - ▶ Accessi ripetuti su stessa pagina non causa un page fault
 - ▶ Esempio, per la sequenza che segue, assumendo 100 bytes per pagina
0100, 0432, 0101, 0612, 0102, 0103, 0104, 0101, 0611, 0102, 0103,
0104, 0101, 0610, 0102, 0103, 0104, 0101, 0609, 0102, 0105
 - ▶ Si ottiene
1, 4, 1, 6, 1, 6, 1, 6, 1, 6, 1
 - ▶ Risultati dipendono dal numero di frame disponibili (se un solo frame allora tutti fault)

Grafo dei Page Fault vs Numero di Frame

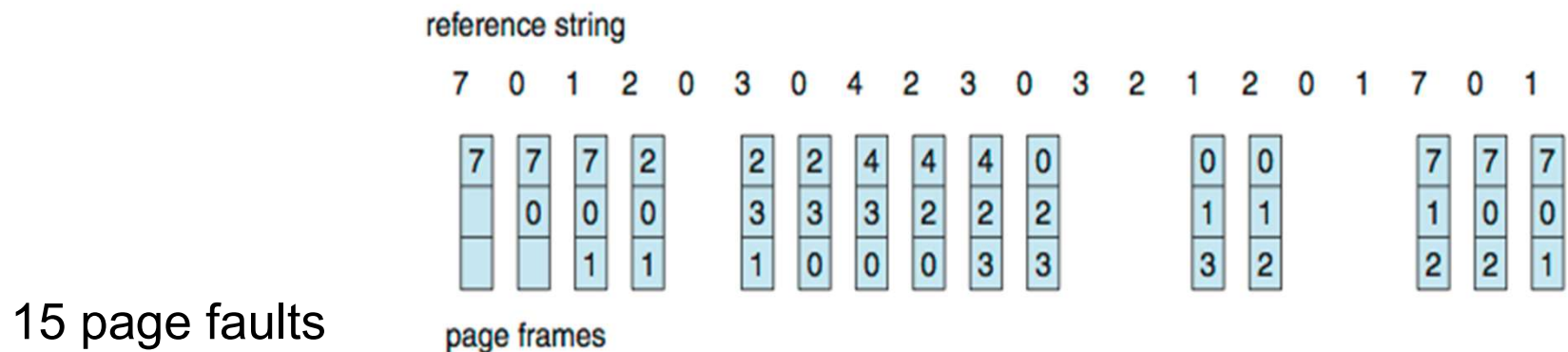


Grafo di page fault per numero di frame
- all'aumentare dei frame diminuiscono i page-fault

In tutti gli esempi che seguono la **stringa dei riferimenti** sarà
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

Algoritmo First-In-First-Out (FIFO)

- Sostituzione della pagina più vecchia
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**
- 3 frame (3 pagine in memoria nello stesso momento per processo)



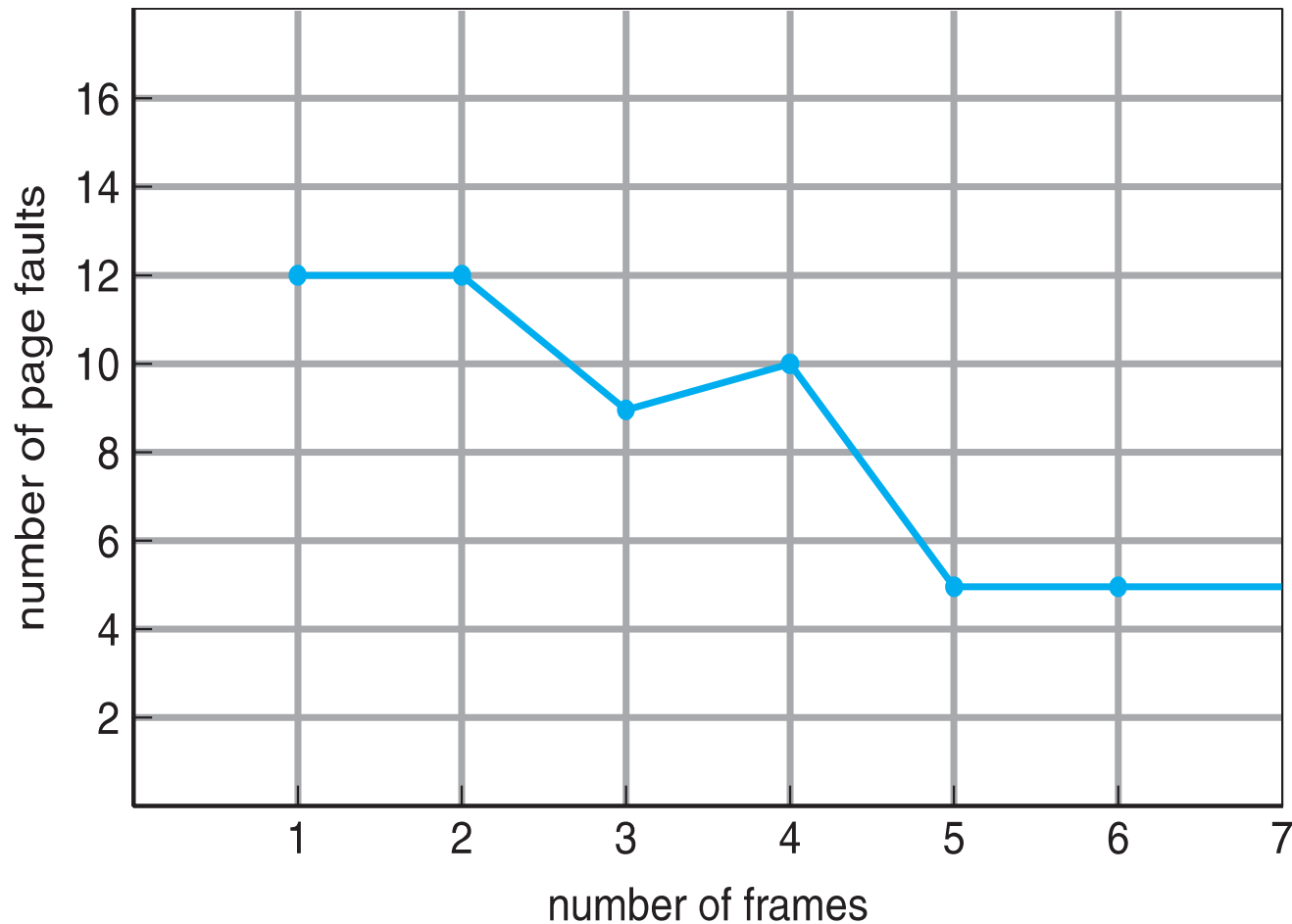
- Le pagine più recenti entrano in coda le più vecchie escono
 - criterio arbitrario (es. vecchie pagine possono contenere dichiarazioni)
- Facile da implementare, ma performance varia con la reference string:
 - Si consideri 1,2,3,4,1,2,5,1,2,3,4,5
 - ... Aggiungendo più frames può causare più page fault
 - ▶ **Anomalia di Belady**

FIFO e anomalia di Belady

Si consideri 1,2,3,4,1,2,5,1,2,3,4,5

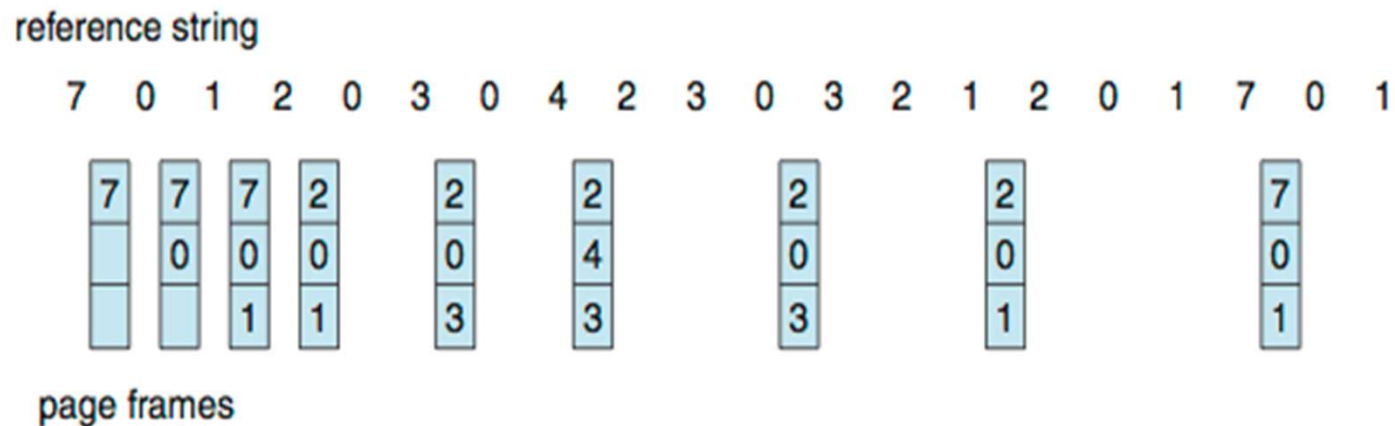
... aggiungere più frames può causare più page fault

Anomalia di Belady (non desiderata)



Algoritmo Ottimale

- ❑ Sostituisci la pagina che non verrà utilizzata per il periodo più lungo
 - ❑ Nell'esempio produce 9 page-fault
- ❑ Come si può sapere?
 - ❑ Non si può leggere nel futuro
- ❑ Usato per valutare le performance degli algoritmi



Algoritmo Least Recently Used (LRU)

- ❑ Usa la conoscenza del passato invece che quella del futuro
- ❑ Sostituisci le pagine che per più tempo non sono state usate
- ❑ Associa ad ogni pagina il tempo dell'ultimo uso

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

- ❑ 12 fault – meglio di FIFO ma peggio di OPT
 - ❑ Con riferimento a 4 sostituisce 2 anche se subito dopo si usa
- ❑ Solitamente un buon algoritmo, frequentemente usato
 - ❑ Ottimale per l'inverso della stringa dei riferimenti
- ❑ Il problema è come implementarlo in modo efficiente ...
 - ❑ assistenza hardware può essere richiesta

Algoritmo LRU

- Due possibili implementazioni
- Implementazione con **Counter**
 - Ogni elemento della tabella delle pagine ha un registro counter (time-of-use)
 - ▶ CPU ha un clock logico/contatore incrementato per riferimenti a memoria
 - ▶ Per ogni riferimento ad una pagina il clock copiato nel counter
 - ▶ Quando una pagina deve essere cambiata, cerca il valore minore del counter
 - ▶ Richiesta: ricerca attraverso la tabella e scrittura del counter per ogni accesso
- Implementazione con **Stack**
 - Mantiene uno stack di numeri di pagina
 - Al riferimento di pagina si mette in cima allo stack
 - ▶ La più recente è in cima, la meno recente sul fondo
 - ▶ Con lista doppiamente linkata facile accedere al fondo e al top
 - ▶ ... non viene fatta alcuna ricerca per la sostituzione
 - ▶ Approccio appropriato per realizzazione (o microcodice) software
- LRU e OPT sono casi di **stack algorithm** senza l'anomalia di Belady
 - Insieme pagine in memoria per n frame sottoinsieme di $n+1$
 - ▶ Per LRU le n più recenti rimangono tali anche con $n+1$ frame

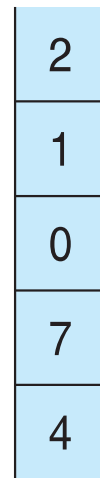
Uso dello stack per registrare il più recente riferimento di pagina

Implementazione con **Stack**

- Mantiene uno stack di numeri di pagina
- Al riferimento di pagina si mette in cima allo stack

reference string

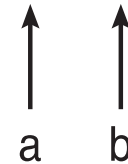
4 7 0 7 1 0 1 2 1 2 7 1 2



stack
before
a



stack
after
b



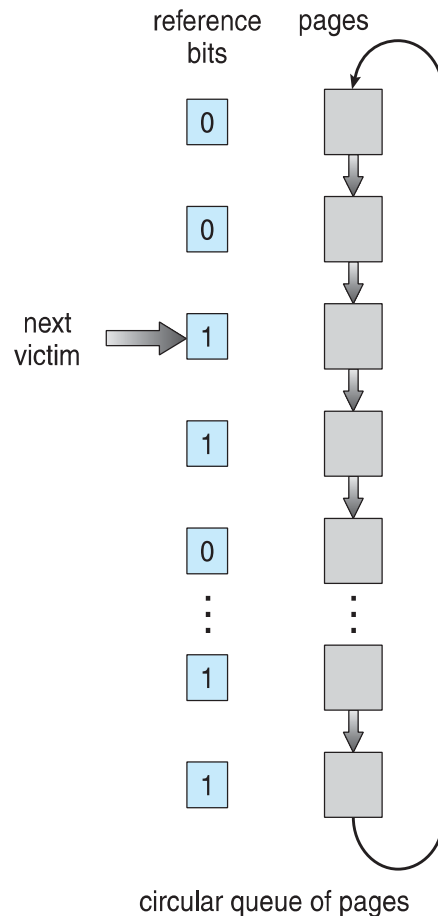
Algoritmi di Approssimazione a LRU

- LRU ha bisogno hardware speciale che non tutti i sistemi forniscono
- In alcuni casi il supporto è un bit di riferimento (**reference bit**)
 - Ogni pagina associata al bit di riferimento, inizialmente = 0
 - Quando si fa riferimento alla pagina (lettura o scrittura) bit settato ad 1
 - Si può sostituire qualunque pagina con bit = 0 (se esiste)
 - ▶ Non si conosce l'ordine di utilizzo ...
- **Algoritmo second-chance**
 - È un FIFO con un reference bit
 - ▶ Dopo la selezione della pagina si controlla il bit, se 1 seconda chance
 - Aggiorna il **clock**
 - Se la pagina da sostituire ha
 - ▶ Reference bit = 0 -> sostituisci
 - ▶ reference bit = 1 allora:
 - setta reference bit 0, lascia la pagina in memoria
 - passa alla prossima pagina con stesse regole
 - Implementato con coda circolare

Second-Chance (clock) Page-Replacement Algorithm

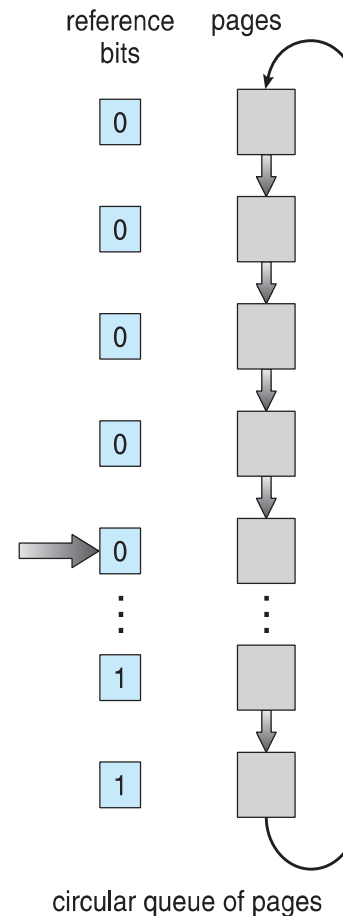
Algoritmo second-chance

- Implementato con coda circolare
- Puntatore indica la pagina da sostituire
- Quando serve un frame avanza finché non trova uno zero
- Pagina sostituita in quella posizione
- Se tutti i bit sono 1 diventa FIFO



circular queue of pages

(a)



circular queue of pages

(b)

Algoritmo Enhanced Second-Chance

- Migliora l'algoritmo usando insieme il reference bit ed il bit di modifica (se disponibile)
- Prendi coppie ordinate (reference, modify)
 1. (0, 0) né usata e né modificata – miglior pagina da sostituire
 2. (0, 1) non usata ma modificata – non così buona, bisogna scrivere prima della sostituzione
 3. (1, 0) usata ma non scritta – potrebbe presto essere usata di nuovo
 4. (1, 1) usata e modificata – può essere riusata e deve essere salvata prima della sostituzione
- Quando occorre una sostituzione di pagina usa lo schema clock (second chance) ma cerca una pagina della classe più bassa non vuota
 - Si potrebbe dover scorrere la lista più volte
 - Preferenza per le pagine che consentono minor I/O

Algoritmi Contatori

- Altri algoritmi per approssimare LRU
- Mantieni un contatore del numero di riferimenti per ogni pagina
 - Non molto usati ... costosi e non approssimano bene l'ottimo
- **Least Frequently Used (LFU) Algorithm**: sostituisci pagine con il contatore più basso
 - Pagine intensamente usate hanno contatori alti
 - Però alcune pagine intensamente usate solo inizialmente
 - ▶ Si può decrementare progressivamente il contatore
- **Most Frequently Used (MFU) Algorithm**: sostituisci pagine con contatore più alto
 - assumendo che le pagine con il contatore più basso siano le più recenti e debbano essere ancora usate

Algoritmi Page-Buffering

- Altri metodi possono essere utilizzati insieme a quelli page-replacement
- I sistemi mantengono un pool di frame liberi
 - Il frame è a disposizione quando occorre, non trovato a tempo di page-fault
 - Leggi la pagina in un frame libero e seleziona una vittima da aggiungere al pool
 - Quando è opportuno porta fuori la vittima
- Variazione: si mantiene lista di pagine modificate
 - Quando il sistema di paging è idle copia pagine modificate in backing store e setta il bit di modifica a non-dirty, si evitano gli accessi in memoria per copiare le pagine modificate
- Si può mantenere intatto il contenuto del frame libero
 - Se è referenziato prima del riuso non occorre ricaricare il contenuto dal disco
 - Alcune versioni di UNIX lo utilizzano con il second chance
 - ▶ Utile per ridurre la penalt  se viene selezionata una vittima sbagliata

Applicazioni e Sostituzione di Pagina

- In tutti gli algoritmi presentati il SO cerca di anticipare il futuro accesso in memoria
- Alcune applicazioni gestiscono meglio – i.e., databases
- Applicazioni che usano intensamente la memoria possono fare buffering, quindi si può avere un doppio buffering
 - SO mantiene copia di pagine in memoria come un I/O buffer
 - L'applicazione mantiene la pagina in memoria per il suo funzionamento
- SO può dare diretto accesso a queste applicazioni
 - **Raw disk** mode e raw I/O
 - Con questa modalità vengono bypassati diversi servizi, es. buffering, file locking, file name, directory, etc.

Allocazione di Frame

- Come si allocano i frame per processo?
- Caso di un sistema con 128 frame
 - SO ne prende 35 e 93 per i processi utente
 - Se pure demand, tutti i 93 nella lista dei frame liberi
 - Il processo in esecuzione richiede i frame
 - ▶ fino a 93 page fault prende pagine libere, poi sostituzione
 - ▶ alla fine rilascio (tutti i 93 nella free frame list)
 - Molte varianti di questo approccio
 - ▶ Si possono sempre lasciare liberi frame sulla frame list e postporre la scrittura nello swap space della vittima per sostituire
 - ▶ Però il metodo alloca tutti i frame per un processo

Allocazione di Frame

- La strategia di allocazione deve tenere conto di un **minimo numero** di frame richiesti da un processo
 - Prestazioni (più frame, meno page fault)
 - Alcune istruzioni richiedono più frame, se page fault restarted
 - ▶ Dipende da architettura
- Esempio: IBM 370 – 6 pagine per l'istruzione SS MOVE:
 - istruzione di 6 byte, può richiedere fino a 2 pagine
 - 2 pagine per gestire *from*
 - 2 pagine per gestire *to*
- Il **massimo** numero dipende dai frame disponibili nel sistema

Allocazione di Frame

- Due principali schemi di allocazione
 - **Fixed allocation**
 - **Priority allocation**
- Molte varianti

Fixed Allocation

□ Equal allocation

- Se m frame per n processi allora alloca ad ognuno m/n
- Per esempio, se 110 frame (tolti quelli per SO) e 5 processi, per ogni processo 20 frame
- I rimanenti mantenuti come free frame buffer pool

□ Proportional allocation

- Però processi diversi (processo studente vs database)
- Alloca proporzionalmente alla dimensione del processo

— s_i = size of process p_i

— $S = \sum s_i$

— m = total number of frames

— a_i = allocation for $p_i = \frac{s_i}{S} \times m$

- Esempio: 64 frame, 2 processi ...

$$m = 64$$

$$s_1 = 10$$

$$s_2 = 127$$

$$a_1 = \frac{10}{137} \times 62 \approx 4$$

$$a_2 = \frac{127}{137} \times 62 \approx 57$$

Fixed Allocation

□ Equal e Proportional allocation

- Se aumenta il numero dei processi diminuisce la memoria allocata per processo
- Problema: stessa allocazione per priorità differenti
 - ▶ Si può includere la priorità nel computo dei frame allocate

□ Priority allocation

- Usa uno schema di proportional allocation usando le priorità invece della dimensione

Rimpiazzamento Locale vs Globale

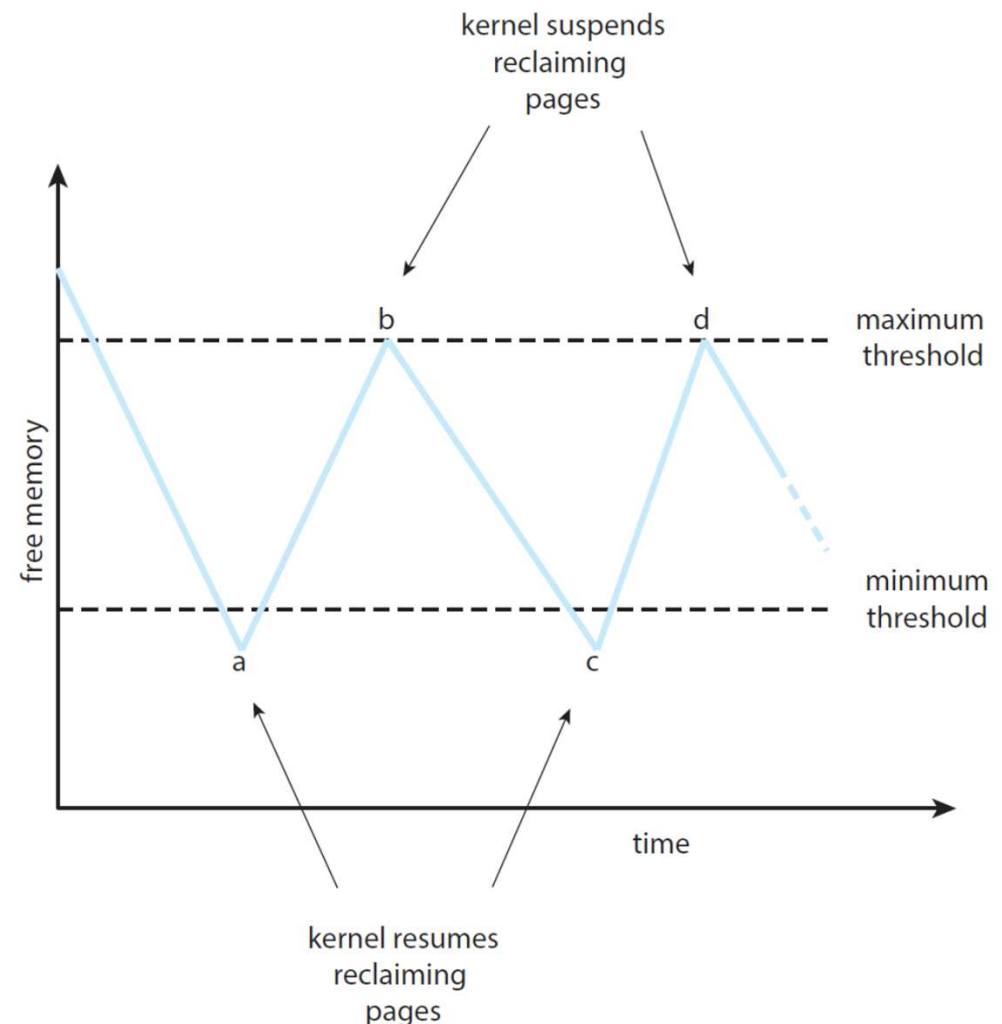
- Se il processo P_i genera un page fault,
 - seleziona per rimpiazzare uno dei suoi frame (locale)
 - selezione per rimpiazzare un frame di un processo con priorità più bassa (globale)

Rimpiazzamento Locale vs Globale

- **Global replacement** – un processo seleziona un frame da rimpiazzare dall'insieme di tutti i frame
 - un processo può prendere un frame di un altro processo
 - ... ma tempi di esecuzione di un processo possono variare molto (dipendono dalle strategie degli altri)
 - ... ma più alto il throughput (quindi scelta più comune)
- **Local replacement** – ogni processo selezione solo dal suo insieme di frame allocati
 - Prestazioni più stabili e consistenti
 - ... ma memoria sottoutilizzata

Allocazione Globale

- ❑ **Global replacement** – un processo seleziona un frame da rimpiazzare dall'insieme di tutti i frame
- ❑ Strategia per global page-replacement
 - ❑ Non si attende che la free-list si esaurisca
 - ❑ Page replacement quando sotto una soglia (tipicamente LRU approx)
 - ❑ Sotto soglia minima il kernel inizia a recuperare frame ...
 - ❑ finché non va sopra la soglia massima
 - ❑ Strategie chiamate **reapers**



Allocazione Globale

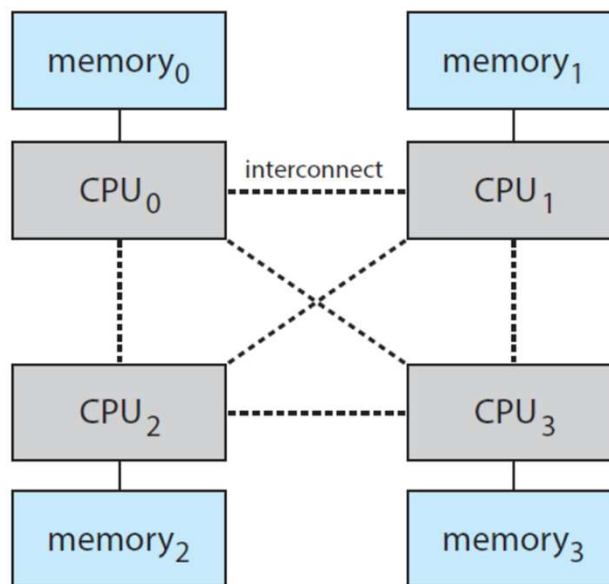
- **Global replacement** – un processo seleziona un frame da rimpiazzare dall'insieme di tutti i frame

- Strategia per global page-replacement può diventare più aggressiva
 - Se non riesce a mantenere i frame liberi può passare da LRU approx a FIFO
 - In Linux se livelli troppo bassi out-of-memory (OOM) killer termina un processo
 - ▶ Ogni processo ha un OOM score, più è alto più rischia
 - ▶ OOM dipende dalla memoria usata, più percentuale più alto il numero
 - ▶ Si può vedere in /proc, es., con PID 2500 /proc/2500/oom_score

- Anche le soglie per attivare le **reaper routine** si possono configurare

Non-Uniform Memory Access

- Fino ad ora si è assunto un accesso uniforme alla memoria virtuale
- Non uniforme con **NUMA** – velocità di accesso a memoria varia
 - Considera schede con più CPU e memorie connesse con bus
 - Ogni CPU con sua memoria locale ad accesso più veloce
 - Accesso in memoria meno veloce ma maggiore parallelismo
 - Se trattato come uniforme l'accesso può essere molto rallentato
 - I frame di memoria allocati più vicino possibile alla CPU di riferimento



Non-Uniform Memory Access

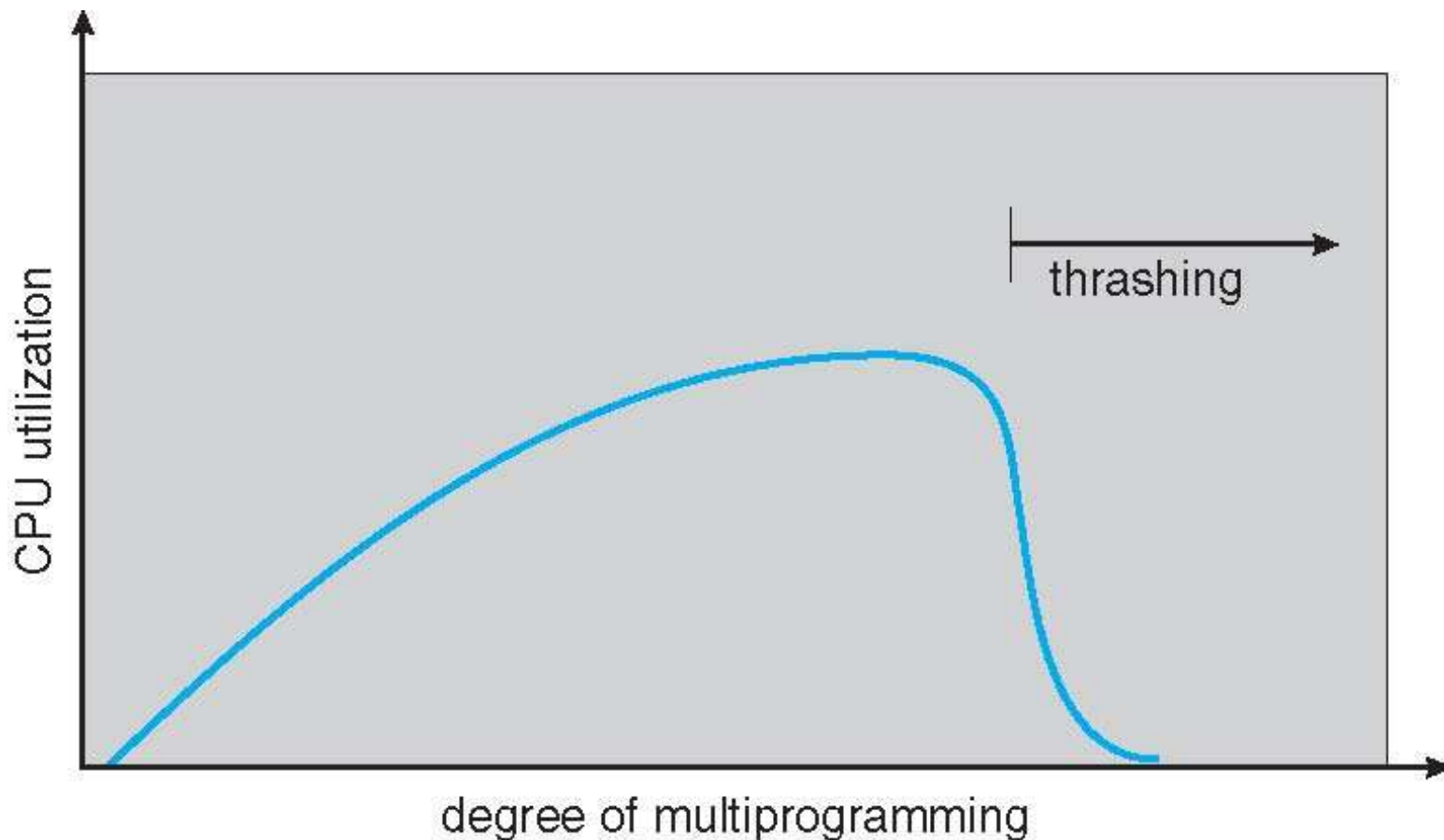
- Fino ad ora si è assunto un accesso uniforme alla memoria virtuale
- Non uniforme con **NUMA** – velocità di accesso a memoria varia
 - allocare memoria “vicina” alla CPU su cui il thread è schedulato
 - se page-fault una memoria virtuale NUMA-aware alloca il nuovo frame vicino
 - Scheduler deve tracciare la CPU su cui gira il processo
 - ▶ allocare stessa CPU e frame vicini, con thread ancora più complicato
- Linux utilizza gerarchia di domini di scheduling
 - ▶ Scheduler limita la migrazione tra domini
 - ▶ Frame-list separate per ogni nodo NUMA
- Solaris utilizza degli **lggroups** (gruppi di località)
 - ▶ Ogni CPU nel gruppo può accedere a memoria nel gruppo con una latenza
 - ▶ Gerarchia di gruppi
 - ▶ Cerca di schedulare i thread di un processo e la memoria per quel processo all'interno del lgroup, altrimenti passa ai gruppi vicini

Thrashing

- ❑ Se un processo non ha “abbastanza” frame, il page-fault rate è molto alto
 - ❑ Page fault per avere una pagina
 - ❑ Rimpiazza un frame esistente
 - ❑ ... ma rapidamente il frame deve essere ripreso
 - ❑ Rimpiazzamento continuo di pagine ...
 - ❑ Questo porta a:
 - ▶ Basso utilizzo di CPU
 - ▶ SO può voler incrementare il grado di multiprogrammazione
 - ▶ Altro processo aggiunto, etc.
- ❑ **Thrashing** $\equiv$ un processo è occupato nello swapping di pagine in e out (più tempo in paging che in esecuzione)

Thrashing

- ❑ SO monitora utilizzo di CPU, se basso aumenta il livello di multiprogrammazione
- ❑ Algoritmo globale di page-replacement, i processi iniziano a sottrarre frame ad altri processi, a loro volta vanno in page-fault, serve il paging device per swap in e out, si accodano i processi e si svuota la coda ready ... aumenta la multiprogrammazione
- ❑ L'utilizzo della CPU aumenta con la multiprogrammazione, poi va in thrashing



Demand Paging e Thrashing

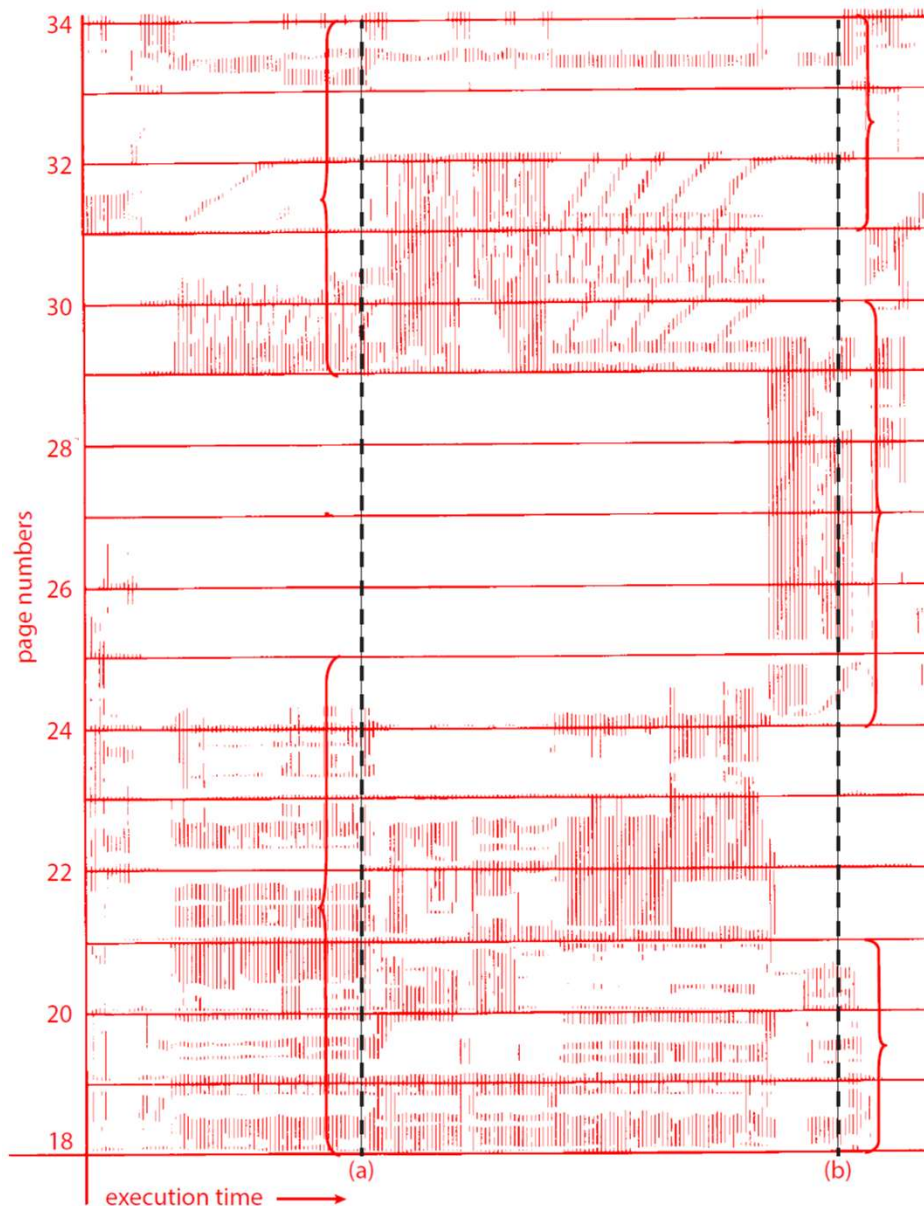
- ❑ Limitare l'effetto del thrashing usando un algoritmo di local replacement
 - ❑ Frame rimpiazzati solo localmente, non possono essere “rubati”
 - ❑ ... un processo in thrashing non trascina gli altri ma ...
 - ❑ ... non risolto il problema, un processo in thrashing occupa il dispositivo di paging e rallenta il page fault di tutti i processi
- ❑ Per prevenire un thrashing bisogna allocare per un processo tanti frame quanti ne necessita, come si fa a sapere?
 - ❑ Guardando a quanti ne sta attualmente usando ...
 - ❑ Si definisce un modello di località (**locality model**) del processo
- ❑ Locality Model:
 - ❑ Un processo passa da locality a locality durante l'esecuzione
 - ❑ Le località sono insiemi di pagine che sono usate insieme
 - ❑ Le località possono sovrapporsi e nuove funzioni corrispondono a nuove località
 - ❑ Località di una funzione: istruzioni, variabili locali, sottoinsieme delle globali

Località in una sequenza di riferimenti in memoria

Località al tempo (a)
{18-24, 29-33}

Località al tempo (b)
{18-20, 24-29, 31-33}

18, 19, 20 si
sovrappongono



Località

- ❑ Le località sono definite dalla struttura del programma e dalle sue struttura dati
- ❑ Notare che il modello di località è anche il principio che giustifica l'utilizzo delle cache
- ❑ Se si riesce ad allocare i frame per le località di un processo non genererà page fault finché non cambia la località
- ❑ Se invece non si allocano frame sufficienti per la località corrente andrà verso il thrashing

Demand Paging e Thrashing

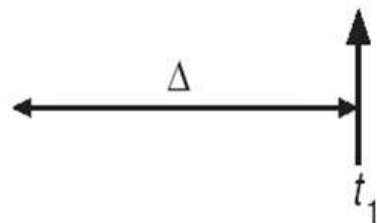
- Si può limitare l'effetto del thrashing usando un algoritmo di local replacement
- Why does demand paging work?
Locality model
 - Process migrates from one locality to another
 - Localities may overlap
- Why does thrashing occur?
 Σ size of locality > total memory size
 - Limit effects by using local or priority page replacement

Modello Working Set

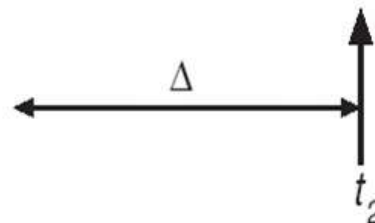
- Il modello del Working Set è basato sull'assunzione di località
- Utilizza parametro Δ per definire una working-set window
 - I più recenti Δ riferimenti in memoria
 - ▶ Es. 10000 istruzioni
- L'insieme delle Δ pagine più recenti è il **Working Set**
 - Se una pagina è in uso attivo è nel WS, se non è più utilizzata esce dal WS dopo Δ unità di tempo dall'ultimo suo riferimento
- Il WS approssima la località di un programma
- Esempio con $\Delta = 10$ (dimensione di WS varia)

page reference table

... 2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 3 4 4 4 3 4 4 4 ...



$$WS(t_1) = \{1, 2, 5, 6, 7\}$$



$$WS(t_2) = \{3, 4\}$$

Modello Working-Set

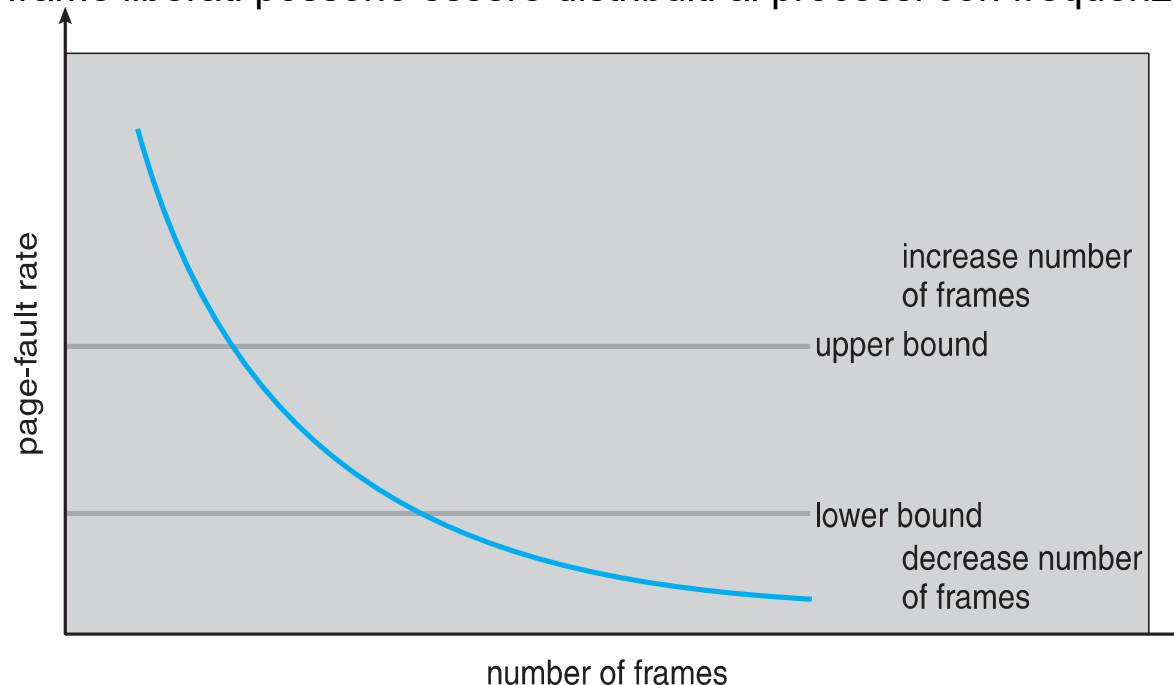
- L'accuratezza del WS dipende da Δ
- Dim del Working Set WSS_i (working set size del processo P_i)
 - numero totale di pagine riferite nel più recente Δ (variabile nel tempo)
 - se Δ troppo piccolo non contiene l'intera località
 - se Δ troppo grande può sovrapporre più località
 - se $\Delta = \infty \Rightarrow$ contiene tutto il programma
- WSS_i è la caratteristica curciale da monitorare
 - $D = \sum WSS_i$ è la richiesta totale di frame
 - Approssima la località
- Se $D > m \Rightarrow$ si verifica il thrashing
- Il SO monitora il Working Set di ogni processo
 - Alloca frame secondo il WSS_i
 - Se ci sono frame extra alloca un nuovo processo
 - Se $D > m$ allora sospende o fa swap out di uno dei processi

Tracciare il Working Set

- La finestra del Working Set è mobile
 - Ad ogni riferimento in memoria un elemento entra, l'altro esce
- Si può approssimare con un timer ad intervalli fissi + un bit di riferimento
- Esempio: per $\Delta = 10000$
 - Timer interrupt dopo ogni 5000 unità di tempo
 - Mantiene in memoria 2 bit per ogni pagina
 - Quando il timer interrompe copia e setta i valori di tutti i reference bit a 0
 - Se bit attuale o uno di 2 bit in memoria = 1 $\Rightarrow$ la pagina è nel working set
 - Non accurato, non sappiamo dove è avvenuto il riferimento in 5000 unità
 - Miglioramento: 10 bit di storia e interrupt ogni 1000 unità di tempo

Frequenza dei Page-Fault

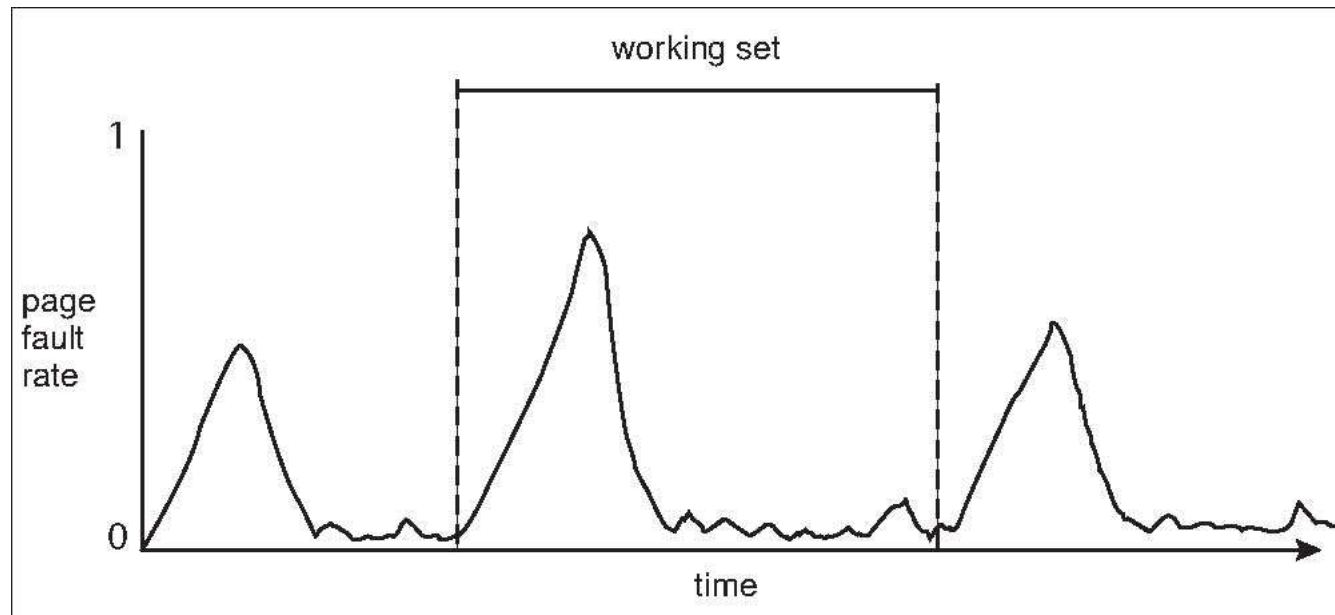
- Approccio più diretto del Working Set Size
- Thrashing aumenta la frequenza di page fault
- Stabilire un tasso “accettabile” di **page-fault frequency (PFF)** per utilizzare una politica di rimpiazzo locale
 - Se il tasso attuale è troppo basso il processo perde frame
 - Se il tasso attuale è troppo alto il processo guadagna frame
- Può essere richiesta la sospensione di un processo
 - i frame liberati possono essere distribuiti ai processi con frequenze più alte



Working Sets e Page Fault Rates

Relazione diretta tra working set di un processo e la frequenza di page-fault

- Working set cambia nel tempo
- Picchi e valli di frequenze di page fault indicano i cabiamenti di località
- L'intervallo tra gli inizi dei picchi definiscono il Working Set



Pratica Corrente

- ❑ Thrashing e swapping hanno un importante impatto sulle prestazioni
- ❑ Il trend attuale prevede l'utilizzo di memoria fisica sufficiente per evitare sia thrashing che swapping
- ❑ Mantenere tutti i working set in memoria tranne che in casi estremi

Compressione di Memoria

- Si utilizza la compressione di memoria per ridurre l'utilizzo di memoria
- Esempio:
 - Free frame list sotto soglia e selezionati frame 15, 3, 35, 26 per liberare memoria

free-frame list

head → 7 → 2 → 9 → 21 → 27 → 16

modified frame list

head → 15 → 3 → 35 → 26

- Invece di scrivere direttamente in swap space fa la compressione di alcuni frame e li mette nei compressed frame (es. 15, 3, 35 compressi in 7 e liberati)

free-frame list

head → 2 → 9 → 21 → 27 → 16 → 15 → 3 → 35

modified frame list

head → 26

compressed frame list

head → 7

- Quando si fa riferimento alle pagine in 7 si decompresse e rialloca

Compressione di Memoria

- Si utilizza la compressione di memoria per ridurre l'utilizzo di memoria
- Usato nei sistemi mobile (Android, iOS), Windows 10 e macOS
- Velocità di compressione vs riduzione di memoria (compression ratio)
 - Maggiore è la velocità minore la compressione
 - Compressione in parallelo su architetture multicore
 - Microsoft Xpress e Apple WKdm sono veloci e garantiscono buone compressioni

Allocare la Memoria per il Kernel

- Trattata in modo differente dalla memoria per processi utente
- Spesso allocate memoria da un pool di free-memory differente
- Due sono le ragioni principali:
 - Kernel richiede memoria per strutture di varie dimensioni, sotto la dimensione della pagina, deve usare la memoria in modo più conservative e mirato
 - La memoria del Kernel in alcuni casi necessita di essere contigua
 - ▶ I.e., dispositivi I/O che dialogano direttamente in memoria fisica, in questi casi la memoria deve essere contigua
- Di seguito si presentano due metodi per l'allocazione di memoria ai processi kernel:
 - Buddy System
 - Allocazione slab

Sistema Buddy

- Alloca memoria da segmenti di dimensioni fissate che consistono di pagine fisicamente contigue
 - Un segmento largo e contiguo suddiviso in porzioni più piccolo; più segmenti combinati rapidamente per ottenere frammento contiguo

- Memoria allocata usando **allocatore di potenza-di-2**
 - Soddisfa richieste in unità di dimensione potenza di 2
 - Richieste approssimate alla più alta Potenza di 2
 - Quando occorre un'allocazione più piccolo della disponibile, il chunk corrente diviso in due buddies della prossima potenza di 2 più bassa
 - ▶ Continua finché un chunk appropriato non è disponibile

- Per esempio, assume chunk di 256KB disponibile, il kernel richiede 21KB
 - Diviso in A_L e A_R di 128KB ognuno
 - ▶ Uno ulteriormente diviso in B_L e B_R di 64KB
 - Uno ulteriormente in C_L e C_R di 32KB ognuno – uno usato per soddisfare la richiesta